Embedded Operating Systems – CDA3631

Final Project Report

Line-following Robot

Jillienne Lapid
April 22, 2026

Table of Contents

Acknowledgement 2

Introduction 3

List of Materials 4

Methods 5

- Block Diagram 5
- Robot Setup 6
- STM32 Board Setup 7
- Key Code Added to main.c 14

Results 18

Discussion 18

Conclusions 18

Github Link 19

References 20

Acknowledgement

I would like to give a special thank you to Dr. Hoan Ngo for his support and guidance through this project. He assisted with materials, insight, and research into completion of the project, as well as taught the topics pertaining to programming and configuring the microcontroller in STM32IDE, key microcontroller concepts, and operating system concepts. With my lack of experience in working with hardware components, his kindness and knowledge were a great help and greatly appreciated.

Furthermore, I would like to thank the Christian Cruz and Christopher Catalano, a team from the Fall 2025 semester, who worked on a similar line-following robot for their documentation and code. Their knowledge and work were used as a reference for understanding how to construct this project. Their work can be found in the following GitHub Repository: https://github.com/Christian10951/Line-Following-Robot-Project

Introduction

The field of robotics has always shown great interest to me due to the combination of software and hardware as well as the logic behind making a robot actionable. With my experience as a computer science major, having taken classes like Operating Systems Concepts and Autonomous Systems Programming, I gained experience working with task management and scheduling in addition to autonomous robotic algorithm design and control theory. Therefore, I wanted to challenge myself to integrate these concepts into real hardware.

This goal of the project was to develop a **Line-following Robot**. This would combine aspects covered in my previous coursework as well as within the Embedded Operating Systems course, such as task handling with FreeRTOS, input and output to pins on the STM32 Board, data handling methods, and control theory.

Through use of Infrared Sensors, a dark-colored line can be distinguished from the table based on the voltage response readings from the sensors. The control system can then parse the incoming data, decide the error, and create commands to drive the motors of the robot forward keeping it in line.

Task scheduling can then be used to manage the continuous collection of data and output of commands to the motors. Mutual Exclusion and Queue structures aided in the communication of data between the tasks. This was managed with the time-critical, real-time operating system, FreeRTOS.

List of Materials

The materials used for this project are a collection of borrowed items provided by the Electrical and Computer Engineering Lab at Florida Polytechnic University as well as some materials personally purchased from online sources.

The total list of materials is listed below, including material name, amount used, source of material, and applicable link to find the material or information about it.

Table 1: List of Materials

Material Name	Amount	Source	Link
STM32 Nucleo F446RE Board	1	FL Poly	https://www.st.com/en/evaluation-tools/nucleo-f446re.html
L298N Motor Driver	1	FL Poly	https://www.handsontec.com/dataspecs/L298N%20Motor%20Driver.pdf
HW-511 Infrared Sensor	3	Amazon	https://www.amazon.com/dp/B0BGYCZ8HV?ref=ppx_yo2ov_dt_b_fed_asin_title
Robot Chassis - **DC Motors** - **Wheels** - **Balancing Joint**	 2 2 1	FL Poly	https://www.dexterindustries.com/store/gopigo3-base-kit/
Mini-Breadboard	1	FL Poly	-
Breadboard M-to-M Wires	~20	FL Poly	-
Breadboard F-to-M Wires	~15	Amazon	https://www.amazon.com/dp/B0BGYCZ8HV?ref=ppx_yo2ov_dt_b_fed_asin_title
Hoovo 2200mAh 11.1V Li-Po Battery	1	FL Poly	-
XT60 Battery Connector	1	Amazon	https://www.amazon.com/dp/B0DNYLFXC9?ref=ppx_yo2ov_dt_b_fed_asin_title&th=1
$5V_{out}$ Power Bank	1	Amazon	https://www.amazon.com/AsperX-10000mAh-Portable-Charging-External/dp/B0B3XB1SCT?th=1

Software used includes STM32CubeIDE v1.19.0 for board configuration and program creation as well as STM32CubeProgrammer v2.22.0 for flashing the final program onto the board for isolated operation.

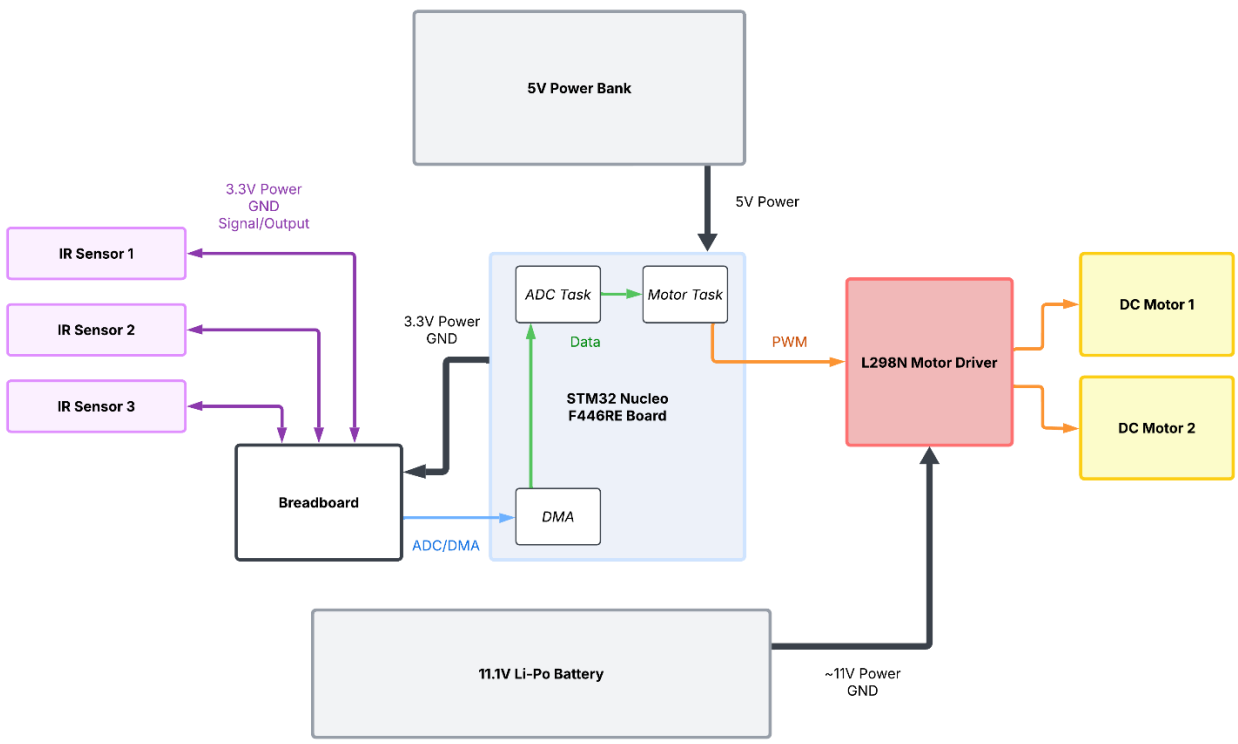
Methods

This section contains a discussion of the setup of the entire system, including a block diagram of component connections and data flow, the layout of the elements on the robot, a description of the setup of the STM32 board pins, and a description of the key code added.

Block Diagram

The Block Diagram below shows a highlevel view of hardware connections and methods used for linking the system.

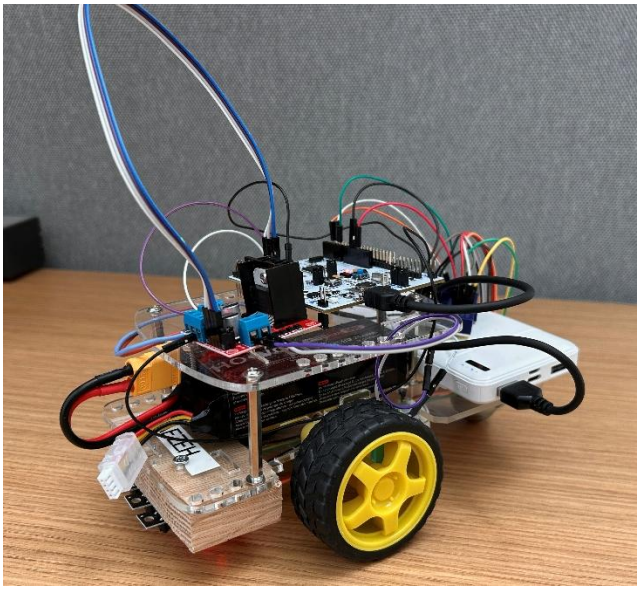
Figure 1: Block Diagram



(Created in LucidChart)

Robot Setup

Figures 2 and 3: Side Views of the Robot



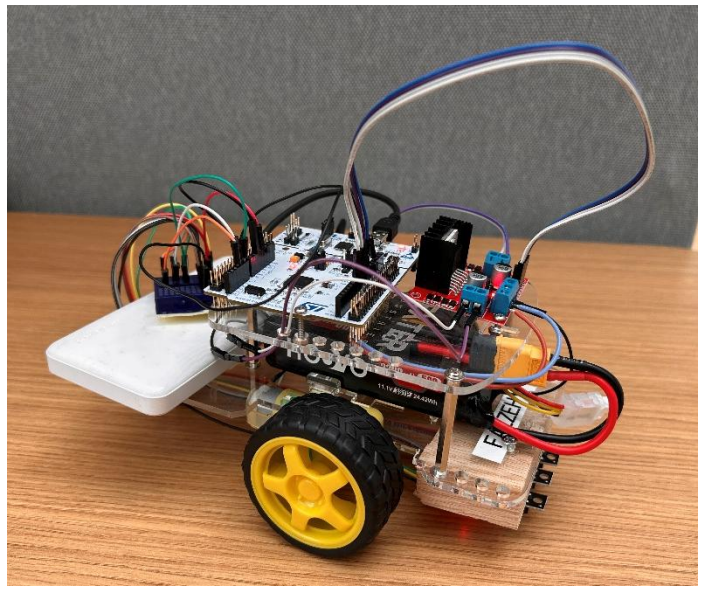


Figure 4: STM32 F446RE Pin Configuration

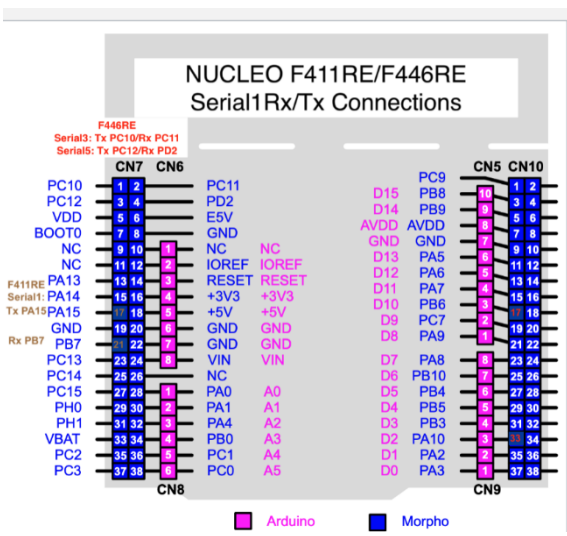
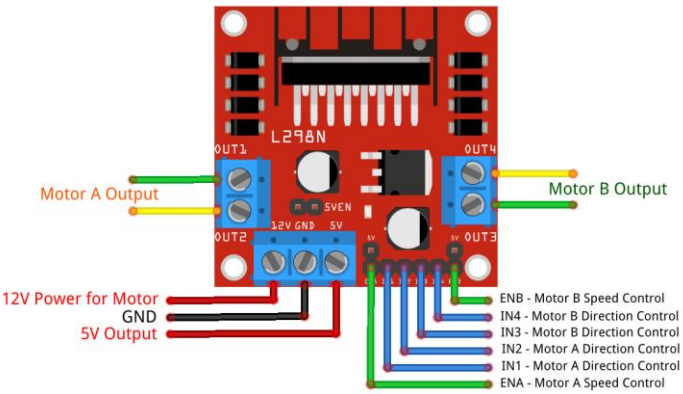


Figure 5: L298N Motor Driver Pin Configuration



The STM32 F446RE Board and Motor Driver were secured to the top level of the robot using mechanical screws and nuts into premade holes in the chassis.

The lower level of the robot held the 11.1V Li-Po Battery, 5V Power Bank, and Breadboard. The placement of the power sources also helped to evenly distribute weight on the robot and make it stable.

The 11.1V Li-Po Battery was used to power the DC Motors and L298N Motor Driver. From the battery to the motor driver, V_{out} was connected to the 12V Pin and the battery ground wire was connected to the GND Pin. Wires were soldered onto the wire-ends of male XT60 Battery Connector to allow connection between the battery and the Motor Driver.

The 5V Power Bank provided power to the STM32 F446RE Board. The power bank had USB Type-A connections, allowing the STM32 link cable to be used. Additionally, a breadboard wire was used to connect the GND Pin on the Motor Driver to a GND pin on the F446RE board.

Breadboard F-to-M wires were used to connect the Motor Driver to the F446RE Board. The chassis was preinstalled with DC motors and female wires for the positive and negative connections. The Motor Driver output pins OUT1-OUT4 were attached to the these female wires where OUT1 and OUT4 are positive, OUT2 and OUT3 are negative.

The Breadboard rested on top of the 5V Power Bank in the back of the robot. The Breadboard served to handle the wiring for the three IR sensors. For each IR sensor, there were three pins associated with them: V_{in}, GND, and S (Signal/Output). Therefore, each sensor needed three F-to-M wires. The V_{in} pins were connected to the 3.3V pin on the F446RE Board. GND on the sensors were connected to GND on the board. Finally, S pins were each connected to pins PA0, PA1, and PA4.

In order to stabilize and attach the three IR sensors, a make-shift piece of hardwood was fixed to the chassis through premade holes. The IR sensors also had premade holes that allowed them to be fixed onto to the hardwood piece. The sensor boards were spaced roughly 5mm apart from one another. To organize the wires, the F-to-M wires were funneled under the chassis, held partially in place by slight tension and the support beams of the balancing joint.

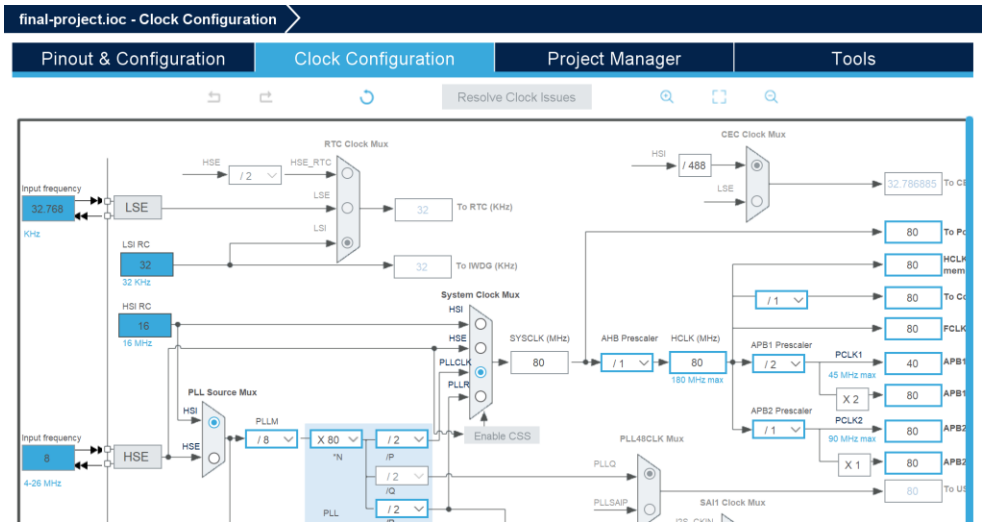
STM32 Board Setup

The STM32 Nucleo F446RE board needs to be configured for I/O, interrupts, analog value reading and handling, and use of a RTOS middleware. This can be done in the .ioc file and by opening the .ioc file within STM32IDE, an interface will appear for easy configuration of the board.

- Clock Configuration

Within the ‘Clock Configuration’ tab, the parameter HCLK was adjusted to 80MHz. This value was selected for smooth operation of system when signal waves were generated to power the motors.

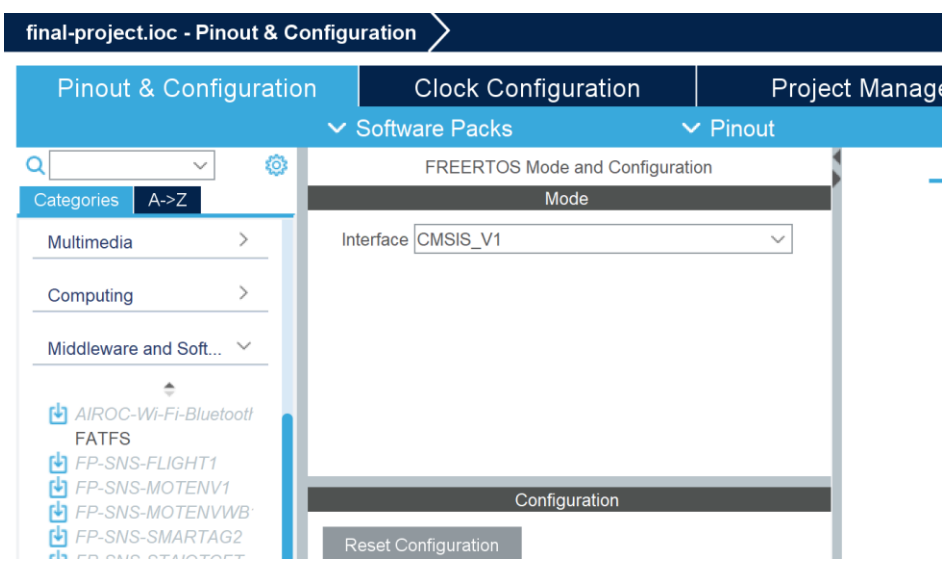
Figure 6: Clock Configuration Tab



- Enabling FreeRTOS

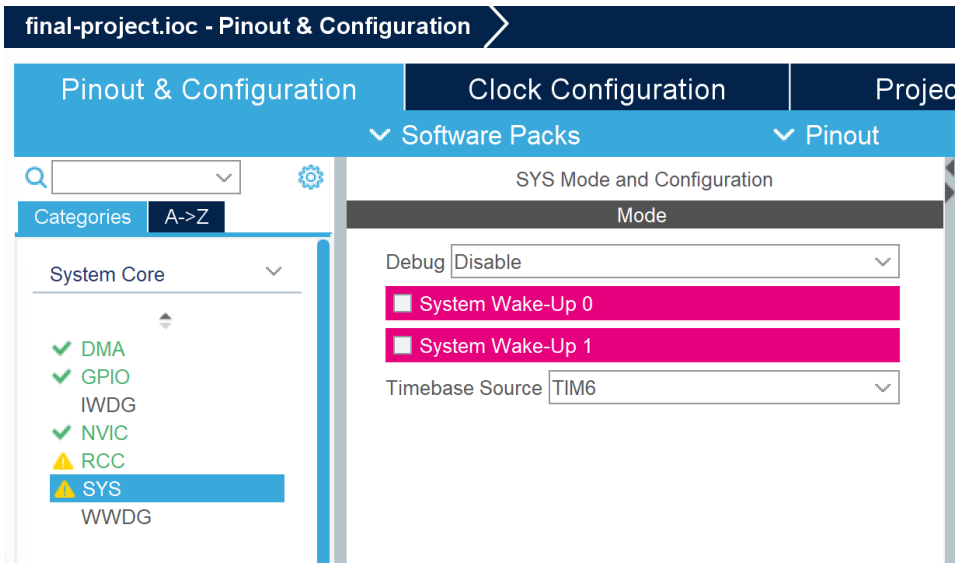
In the ‘Pinout & Configuration’ tab, under the category ‘Middleware and Software Packs’, scroll down to “FREERTOS”. To enable it, change the interface to “CMSIS_V1”. An initial task, called defaultTask, will be created upon enabling the interface. This can be ignored later on in the main.c file, and its code contents deleted as it will not be used.

Figure 7: Changing Interface of FreeRTOS to Activate It



Due to enabling FreeRTOS, the Timebase source of the system must be changed. Typically, SysTick is used by the system but this will now be used by FreeRTOS. Under the category 'System Core' and within the 'SYS' page, change 'Timebase Source' to a different timer, such as TIM6 which was selected for this project.

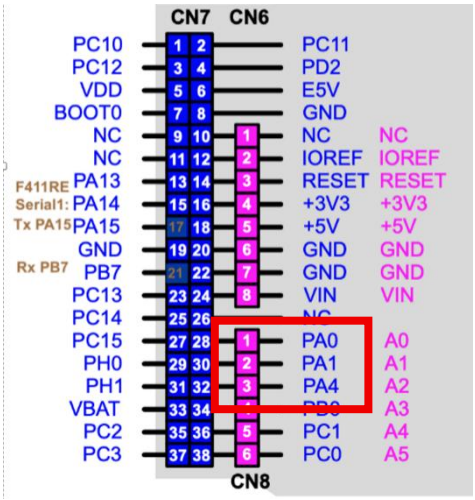
Figure 8: Changing the Timebase Source to TIM6 (Timer 6)



- Enabling ADC for Three Pins

  To read the data from three IR sensors, three pins were configured for ADC input. The selected pins were PA0, PA1, and PA4 as they are located in succession on the STM32 F446RE Board and are nearby the 3.3V output pin.

Figure 9: Pins PA0, PA1, and PA4 on Left-side of F446RE Board



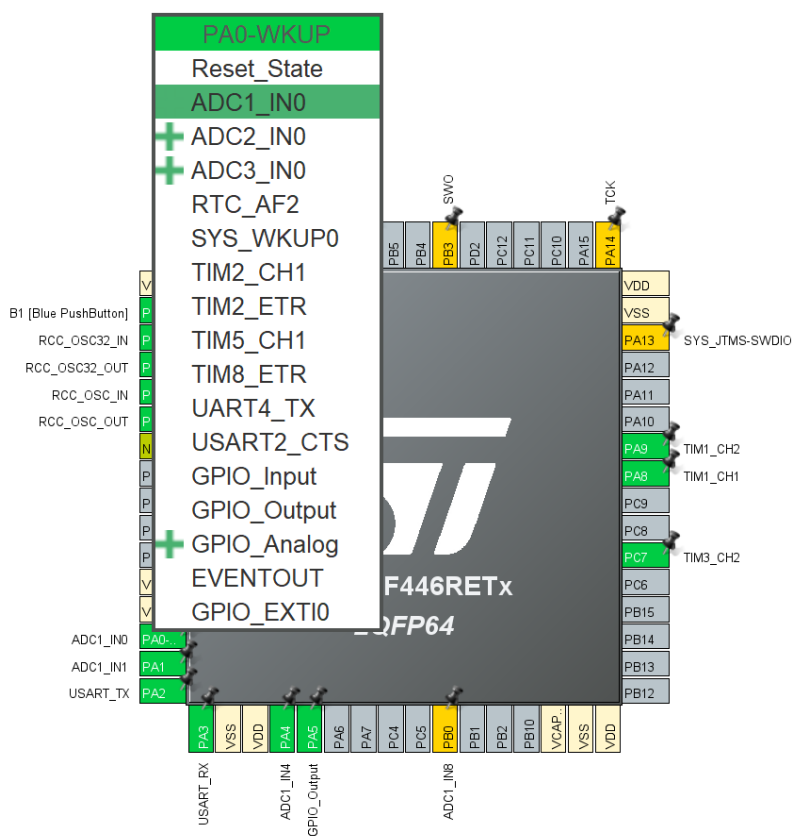
The signal, or S pins on the IR sensors were connected to the three aforementioned pins via F-to-M breadboard wires.

Table 2: IR Sensor Pin Mapping

F446RE Pin	IR Sensor + Pin	
PA0	Right Sensor	IR1, S pin
PA1	Middle Sensor	IR2, S pin
PA4	Left Sensor	IR3, S pin

Each pin made use of ADC1. Within the 'Pinout view' interface, PA0 was set to 'ADC_IN0', PA1 to 'ADC_IN1', and PA4 to 'ADC_IN4'.

Figure 10: Configuring PA0, PA1, and PA4 to input to ADC1



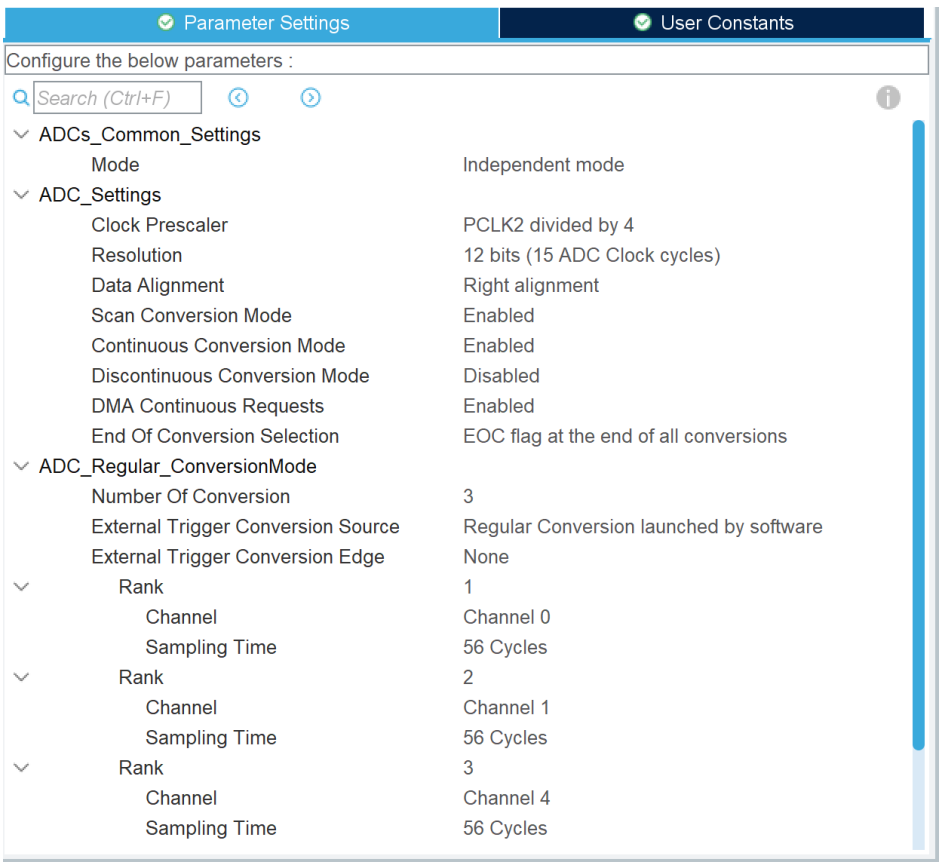
Then, under 'Pinout & Configuration' and category 'Analog' on page ADC1, the following parameters in 'Parameter Settings' were adjusted:

Tables 3 and 4: ADC1 Parameter Settings

ADC_Settings	
Scan Continuous Conversion Mode	Enabled
Continuous Conversion Mode	Enabled
DMA Continuous Requests	Enabled

ADC_Regular_ConversionMode	
Number of Conversions	3
Rank 1	Channel: 0, 1, 4 (respectively) Sampling Time: 56 Cycles
Rank 2	
Rank 3	

Figure 11: Full List of ADC1 Parameters

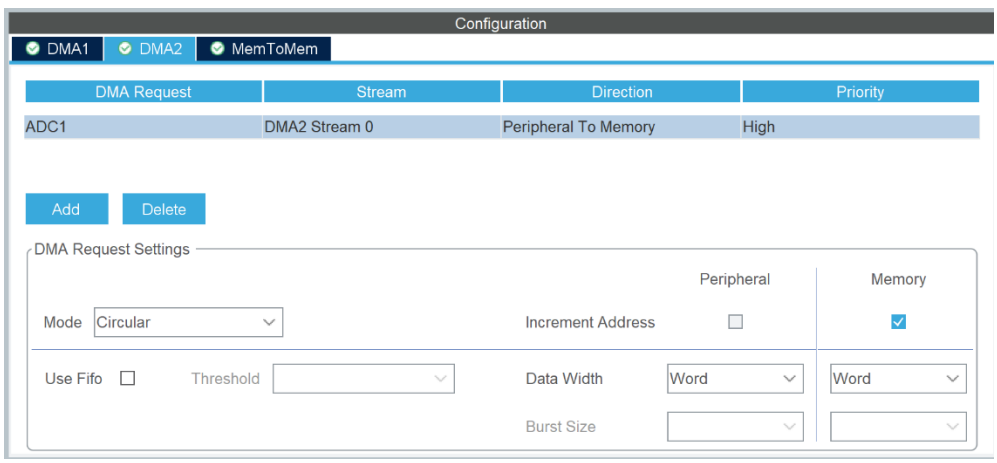


- Enabling DMA for ADC values

  For reading of the ADC data, a DMA connection was established. In the tab 'Pinout & Configuration' , under category 'System Core, and in page 'DMA', DMA2 had an added configuration to request from ADC1, stream to Stream 0, have direction of peripheral to memory, and have high priority.

  Additionally, the in the 'DMA Request Settings' section, the mode of DMA was configured to "Circular." Furthermore, the 'Data Width' parameter was changed from "HALFWORD" to "WORD" in order to account for the size of data coming from the read analog values being stored.

  Figure 12: DMA2 Configuration



  This could also be performed in the ADC1 configuration under subtab 'DMA Settings.' Ensure that the parameter 'DMA Continuous Requests' was enabled, as mentioned in the previous section about ADC setup.

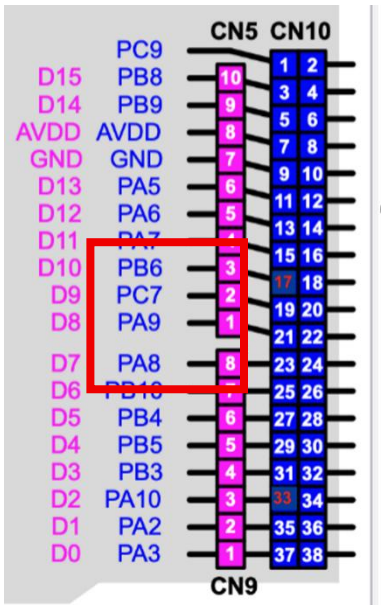
- Enabling PWM generation on Four Pins

  To send controlled speed commands to the DC motors, PWM signals were used. There were two motors used which were connected to the L298N Motor Driver. This has four pins to send output to the motors, labeled IN1, IN2, IN3, and IN4. IN1 controls forward rotation while IN2 controls backward rotation for the left wheel; IN3 and IN4 behave similar for the right wheel.

  I selected pins PA8, PA9, PC7, and PB6 due to their position in succession on the board and so as to not interfere with the left-side column of pins.

Under ‘Pinout & Configuration’ in the ‘Pinout view’ interface, the pins listed above were configured to be associated with a specific timer and channel on the timer.

Figure 13: PA8, PA9, PC7, and PB6 on the Right-side of the F446RE Board



The pins were connected using F-to-M breadboard wires.

Table 5: Motor Driver Pin Mapping

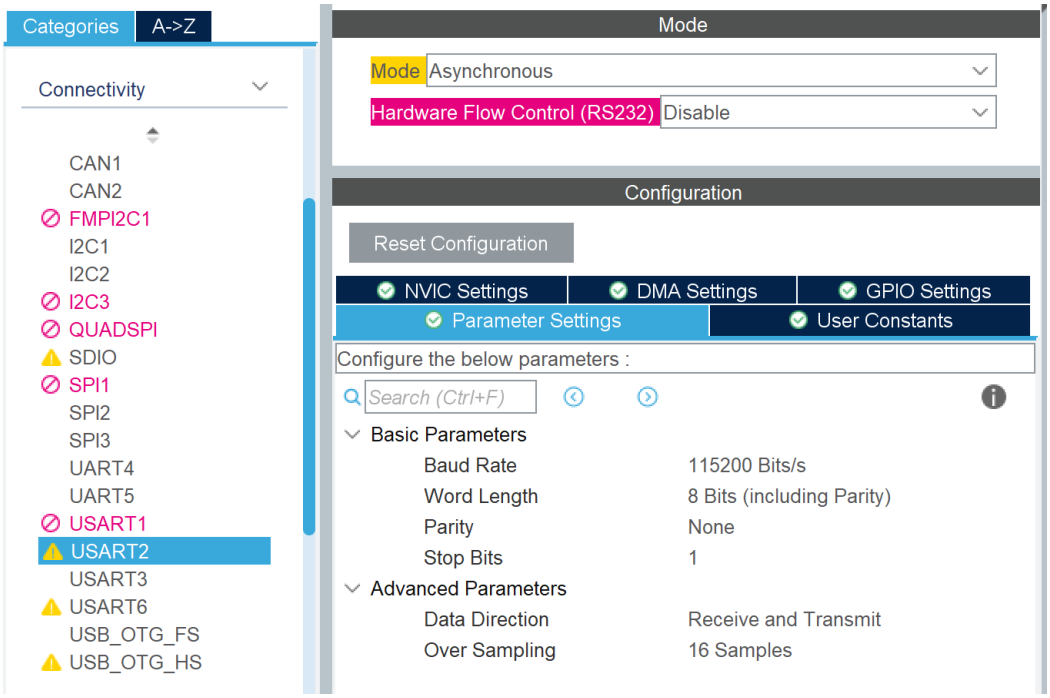
F446RE Pin	Wheel + Motor Driver Pin	
PA8	Right, Forward	IN1 Pin
PA9	Right, Backward	IN2 Pin
PC7	Left, Forward	IN3 Pin
PB6	Left, Backward	IN4 Pin

- Ensuring UART Connection

  Last, mainly for debugging purposes, a UART connection was used to print out console messages. This was used for testing proper receiving and sending out of data and proper hardware functionality.

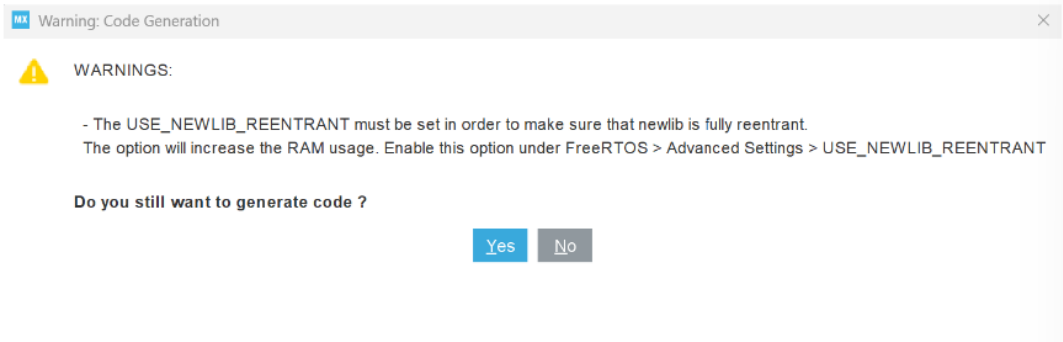
  In ‘Pinout & Configuration,’ under the category ‘Connectivity,’ and in the page USART2, the connection was ensured to be in Asynchronous mode to ensure UART communication. The default parameters were used, including a baud rate of 115,200 Bits/s.

Figure 14: UART2 Configuration for UART Connection



Upon saving the .ioc file, code will be generated in the main.c file that corresponds to the changes made in the interface described above. A warning message will appear due to changing the Timebase Source to a different value (from SysTick to a different timer). If the warning appears as so, it is safe to proceed and allow the code generation to occur.

Figure 15: Warning Message Due to Changing the Timebase Source



Key Code Added to main.c

After code generation after saving the .ioc file, additional aspects such as constants, tasks, and semaphores were added into the main.c file afterwards.

- Libraries

Figure 16: Includes

Under the 'Private Includes' header, the string, standard I/O, and task libraries were included.

```
/* USER CODE END Header */
/* Includes ------------------------
#include "main.h"
#include "cmsis_os.h"

/* Private includes ----------------
/* USER CODE BEGIN Includes */
#include "string.h"
#include "stdio.h"
#include "task.h"
/* USER CODE END Includes */
```

- ADC/DMA Buffer

Figure 17: Array acting as ADC Buffer

Under the 'Private Variables' header, a buffer of length 3 with storage of data of type uint32 was defined for use of ADC and DMA for reading incoming analog values.

```
/* Private variables ---------------------------------
ADC_HandleTypeDef hadc1;
DMA_HandleTypeDef hdma_adc1;

TIM_HandleTypeDef htim1;
TIM_HandleTypeDef htim3;
TIM_HandleTypeDef htim4;

UART_HandleTypeDef huart2;
/* USER CODE BEGIN PV */

// Buffer for ADC to store the 3 sensor input values
#define ADC_BUFFERLEN 3
uint32_t adcBuffer[ADC_BUFFERLEN];
```

- Task and Semaphore Definitions

Figure 18: Defining the Tasks, Task functions, and Semaphore

A task was created for handling the control of the robot, including reading the ADC values and parsing through them

A semaphore was defined for protecting the accessing of data from the DMA

```
/* Private user code ---------------------
/* USER CODE BEGIN 0 */
xTaskHandle StartControlTaskHandler;
void StartControlTask(void *argument);

SemaphoreHandle_t xDMASemaphore;

/* USER CODE END 0 */
```

- Task and Semaphore Creation

Figure 19: Creating the Tasks and Semaphore

```
/* USER CODE END 2 */

/* USER CODE BEGIN RTOS_MUTEX */
/* add mutexes, ... */
/* USER CODE END RTOS_MUTEX */

/* USER CODE BEGIN RTOS_SEMAPHORES */
xDMASemaphore = xSemaphoreCreateBinary();

/* USER CODE END RTOS_SEMAPHORES */

/* USER CODE BEGIN RTOS_TIMERS */
/* start timers, add new ones, ... */
/* USER CODE END RTOS_TIMERS */

/* USER CODE BEGIN RTOS_QUEUES */

/* USER CODE END RTOS_QUEUES */

/* Create the thread(s) */
/* definition and creation of defaultTask */

/* USER CODE BEGIN RTOS_THREADS */
xTaskCreate(StartControlTask, "Control", 512, NULL, 3, &StartControlTaskHandler);

/* USER CODE END RTOS_THREADS */
```

Using xTaskCreate() and xSemaphoreCreateBinary() the aforementioned task and semaphore were created.

- ADC Conversion Completed Callback

Figure 20: Defining the function that is called upon ADC Read completion

```
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
{
    if (hadc->Instance == ADC1)
    {
        BaseType_t xHigherPriorityTaskWoken = pdFALSE;
        xSemaphoreGiveFromISR(xDMASemaphore, &xHigherPriorityTaskWoken);
        portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    }
}
```

This function is called at the end of the ADC conversion operation. Upon the data being read, the semaphore gives up control and allows for the rest of the task that uses the semaphore to continue.

- StartControlTask() Defintion

Figure 21: Logic for the StartControlTask() function

```c
void StartControlTask(void *argument)
{
    HAL_ADC_Start_DMA(&hadc1, adcBuffer, ADC_BUFFERLEN);

    HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
    HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2);
    HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_2);
    HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_1);

    char msg[80];
    uint16_t IR_right, IR_mid, IR_left;

    #define LINE_THRESHOLD  800    // below = no line, above = line detected
    #define BASE_SPEED      400
    #define TURN_SPEED      200    // speed of the slower wheel during a turn

    for (;;)
    {
        if (xSemaphoreTake(xDMASemaphore, portMAX_DELAY) == pdTRUE)
        {
            // --- 1) Read IR data ---
            taskENTER_CRITICAL();
            IR_right = (uint16_t)adcBuffer[0];   // IR1 = right
            IR_mid   = (uint16_t)adcBuffer[1];   // IR2 = middle
            IR_left  = (uint16_t)adcBuffer[2];   // IR3 = left
            taskEXIT_CRITICAL();

            uint8_t see_right = (IR_right > LINE_THRESHOLD);
            uint8_t see_mid   = (IR_mid   > LINE_THRESHOLD);
            uint8_t see_left  = (IR_left  > LINE_THRESHOLD);

            // --- 2) Print IR data ---
            snprintf(msg, sizeof(msg), "R:%u M:%u L:%u  [%c%c%c]\r\n",
                     IR_right, IR_mid, IR_left,
                     see_right ? 'X' : '.',
                     see_mid   ? 'X' : '.',
                     see_left  ? 'X' : '.');
            HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), 100);

            if (see_mid && !see_left && !see_right)
            {
                // Line is centered — go straight
                __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, BASE_SPEED);
                __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, BASE_SPEED);
            }
            else if (see_right && !see_left)
            {
                // Line drifted right — turn right (slow left wheel, fast right)
                __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, BASE_SPEED);
                __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, TURN_SPEED);
            }
            else if (see_left && !see_right)
            {
                // Line drifted left — turn left (fast left wheel, slow right)
                __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, TURN_SPEED);
                __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, BASE_SPEED);
            }
            else if (see_left && see_right && !see_mid)
            {
                // Both outer sensors on line (junction or sharp bend) — go straight
                __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, BASE_SPEED);
                __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, BASE_SPEED);
            }
            else
            {
                // No line detected at all — stop and wait
                __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, 0);
                __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 0);
            }

            vTaskDelay(pdMS_TO_TICKS(50));
        }
    }
}
```

This function Starts the DMA and the PWM signals. The signals were started for all pins, but only the forward motion Timers were implemented.

The semaphore protects the reading of the ADC values from the adcBuffer[].

Values are filtered to see if they are above a LINE_THRESHOLD, which I defined to be above 800. When detecting the line, the ADC reading is above 2000, so this helps to filter the values off the bat.

For debugging the sensor data is printed out.

Conditional statements are then used to determine based on which sensors see the line, what speed to assign to the left and right motors.

Results

The robot ended up successfully being able to follow a line, keeping good speed and adjusting the speed of the motors to traverse tighter turns and keep on the line. The setup of the robot was also successful with all parts acting as intended and being able to fit in a reasonable manner on the robot.

Discussion

The intention of the project was to develop a proportionally controlled robot. I started with implementing a PID controller and then a PD controller, but ran into issues with both. Due to time constraints, I defaulted to using simpler logic by using conditions and the values read from the sensors to determine what value to send to the motors. This ended up still being an effective solution as it achieved the goals of line-following and may have worked better due to my setup with only three IR sensors.

Additionally, the mount I had to use for the sensors was very last-minute and makeshift, using hardwood and wood screws to secure the sensors in place. This made iteration of the mount design and adjustment difficult.

Ideally, with a larger time frame, I would have liked to do the following:

- Implement a PID controller
- Use more IR sensors for more data to work with
- Use more permanent wiring solutions for a cleaner build
- 3D Print mounts for the board, sensors, and components

Conclusions

In all, this project was very rewarding and fun for me. I was able to work with hardware and implement software concepts that I have learned in my previous classes. Additionally, I implemented some of the operating systems concepts with scheduling, task creation, and mutual exclusion protection of critical data. It was exciting to see the logic I built in physical form and achieving the main goal I set for the project. In the future, I would like to build the control system and make a more polished system. These are possible goals that I can see myself working towards in a future iteration of this project.

Github Link

The link to the remote GitHub repository is listed below:

https://github.com/jilliennelapid/Line-Following-PID-Robot-Project

This repository contains the following information:

README.md	README file describing the project
Final-Project-Report_Jillienne-Lapid.pdf	This report as a PDF
Final-Project-Slides_Jillienne-Lapid.pdf	The slideshow for this project as a PDF
final_project.c	The main.c file code
Final-Project-Lapid-Video.txt	A text file with a link to the YouTube video demo

References

Christian Cruz and Christopher Catalano, 2025

https://github.com/Christian10951/Line-Following-Robot-Project

STM32 Nucleo-64 Boards - User Manual

https://www.st.com/resource/en/user_manual/dm00105823.pdf

L298N Motor Driver User Guide

https://www.handsontec.com/dataspecs/L298N%20Motor%20Driver.pdf

STM32CubeIDE Download

https://www.st.com/en/development-tools/stm32cubeide.html

STM32CubeProgrammer Download

https://www.st.com/en/development-tools/stm32cubeprog.html